JavaScript Events

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Contexte 3

II. Les événements 3

A. La définition d'un événement....3
B. Supprimer un gestionnaire d'événements....4
C. Les différents événements....4
D. Créer ses propres événements....5
E. Exercice : Quiz....6

III. Le fonctionnement avancé des événements 7

A. L'objet Event....7
B. La propagation des événements....8
C. La capture d'événements....9
D. Empêcher la propagation d'événements....10
E. PreventDefault....11
F. Exercice : Quiz....11

IV. Essentiel 12

V. Auto-évaluation 12

A. Exercice....12
B. Test....14

Solutions des exercices 15

Tous droits réservés à STUDI - Reproduction interdite

I. Contexte

Durée : 1 heure

Environnement de travail : VS CODE

Contexte

La programmation événementielle en JavaScript est une approche qui permet de gérer les événements déclenchés par l'utilisateur ou par le système, en leur attribuant des actions à réaliser en réponse. Cette approche est couramment utilisée dans le développement web pour créer des interfaces utilisateur interactives.

La programmation événementielle en JavaScript peut sembler complexe, mais en réalité elle est assez simple à comprendre. Il suffit de lier des fonctions à des événements pour leur donner une action à réaliser en réponse. Un événement est donc lié à un code à exécuter. JavaScript fournit des fonctions intégrées pour la gestion des événements, ce qui facilite grandement le travail des développeurs.

Dans ce cours, nous allons explorer les bases de la programmation événementielle en JavaScript, en expliquant les concepts clés tels que les événements, les écouteurs d'événements et la façon de les utiliser pour créer des interfaces utilisateur interactives.

II. Les événements

A. La définition d'un événement

Définition

Les événements sont des actions qui se produisent sur une page web, tels que le clic d'un bouton ou la soumission d'un formulaire. En JavaScript, nous pouvons détecter ces événements et y répondre en exécutant une fonction. Les événements peuvent être déclenchés par l'utilisateur, le navigateur ou le code JavaScript lui-même.

Il existe de nombreux types d'événements en JavaScript, tels que le clic, le chargement de la page, la soumission de formulaire, le déplacement de la souris, etc. Chaque type d'événement est associé à un objet Event qui contient des informations sur l'événement, telles que la cible de l'événement (l'élément HTML qui a déclenché l'événement), le type d'événement et les données spécifiques à l'événement.

Des événements se produisent en permanence sur une page web, mais si on veut exécuter du code après un événement, on doit « *écouter un événement* ».

Méthode

Pour écouter un événement en JavaScript, il est possible d'utiliser la méthode `addEventListener()`. Cette méthode prend 2 arguments : le type d'événement à écouter et la fonction à exécuter lorsque l'événement se produit. Voici un exemple de code qui écoute le clic d'un bouton et exécute une fonction lorsque le clic se produit :

```
1 const button = document.querySelector('button');
2
3 button.addEventListener('click', () => {
4   alert('Bouton cliqué');
5 });
```

Dans cet exemple, la méthode `querySelector()` est utilisée pour sélectionner le premier élément de type « *button* » dans le document HTML. Ensuite, la méthode `addEventListener()` est utilisée pour écouter le clic de ce bouton et exécuter une fonction anonyme qui affiche une alerte avec le texte « *Bouton cliqué* ».

B. Supprimer un gestionnaire d'événements

Méthode

La méthode `removeEventListener()` est utilisée pour supprimer un gestionnaire d'événements d'un élément. Cette méthode prend également 2 arguments : le type d'événement et une fonction de gestionnaire d'événements.

```
const button = document.querySelector('button');

function handleClick(event) {
  console.log('Bouton cliqué');
}

button.addEventListener('click', handleClick);

// Supprime le gestionnaire d'événements handleClick
button.removeEventListener('click', handleClick);
```

Dans ce code, si on clique sur le bouton, la méthode `handleClick` ne sera pas appelée, car le `removeEventListener` annule le `addEventListener`.

C. Les différents événements

Événement	Description
click	Déclenché lorsque l'utilisateur clique sur un élément de la page.
keyup	Déclenché lorsque l'utilisateur relâche une touche de clavier.
submit	Déclenché lorsque l'utilisateur soumet un formulaire.
load	Déclenché lorsque la page web a fini de charger.
mouseover	Déclenché lorsque le curseur de la souris se déplace sur un élément.
mouseout	Déclenché lorsque le curseur de la souris quitte un élément.
scroll	Déclenché lorsque l'utilisateur fait défiler la page.
resize	Déclenché lorsque la taille de la fenêtre du navigateur est modifiée.
contextmenu	Déclenché lorsque l'utilisateur fait un clic droit sur un élément.

Tous droits réservés à STUDI - Reproduction interdite

Événement	Description
mousedown	Déclenché lorsque l'utilisateur enfonce un bouton de la souris sur un élément.
mouseup	Déclenché lorsque l'utilisateur relâche un bouton de la souris sur un élément.
touchstart	Déclenché lorsque l'utilisateur touche l'écran tactile.
touchend	Déclenché lorsque l'utilisateur relâche l'écran tactile.
touchmove	Déclenché lorsque l'utilisateur déplace son doigt sur l'écran tactile.
error	Déclenché lorsqu'une erreur se produit lors du chargement de la page ou d'une ressource.
change	Déclenché lorsqu'un élément de formulaire (ex. : input) est modifié.

Cette liste n'est pas exhaustive, mais elle donne une idée des événements les plus couramment utilisés en JavaScript. Les développeurs peuvent également créer leurs propres événements personnalisés pour répondre aux besoins spécifiques de leur application.

D. Créer ses propres événements

Outre les événements intégrés à JavaScript, il est également possible de créer des événements personnalisés pour répondre aux besoins spécifiques de votre application. Les événements personnalisés peuvent être déclenchés à n'importe quel moment du code, et les écouteurs d'événements peuvent leur être attachés de la même manière que pour les événements intégrés.

Méthode

Pour créer un événement personnalisé en JavaScript, vous devez utiliser l'objet `CustomEvent`. Voici un exemple de code qui crée un événement personnalisé nommé monEvenement :

```
// Création de l'événement personnalisé
const monEvenement = new CustomEvent('monEvenement');

// Ajout d'un écouteur d'événement pour l'événement personnalisé
document.addEventListener('monEvenement', () => {
  console.log('L\'événement personnalisé a été déclenché !');
});

// Déclenchement de l'événement personnalisé
document.dispatchEvent(monEvenement);
```

Tous droits réservés à STUDI - Reproduction interdite

Dans cet exemple, nous avons créé un objet `CustomEvent` avec le nom monEvenement. Ensuite, nous avons ajouté un écouteur d'événement pour cet événement personnalisé en utilisant la méthode `addEventListener` de l'objet document. Finalement, nous avons déclenché l'événement personnalisé en utilisant la méthode `dispatchEvent` de l'objet document.

Complément

Il est également possible de passer des données avec l'événement personnalisé en utilisant l'objet `CustomEvent`. Voici un exemple de code qui crée un événement personnalisé nommé monEvenement avec des données :

```
// Création de l'événement personnalisé avec des données
const monEvenement = new CustomEvent('monEvenement', {
  detail: {
    message: 'Bonjour tout le monde !'
  }
});

// Ajout d'un écouteur d'événement pour l'événement personnalisé avec des données
document.addEventListener('monEvenement', (event) => {
  console.log(`Message reçu : ${event.detail.message}`);
});

// Déclenchement de l'événement personnalisé avec des données
document.dispatchEvent(monEvenement);
```

Dans cet exemple, nous avons créé un objet `CustomEvent` nommé monEvenement avec l'option detail pour y inclure des données. Ensuite, nous avons ajouté un écouteur d'événement pour cet événement personnalisé en utilisant la méthode `addEventListener` de l'objet document. Finalement, nous avons déclenché l'événement personnalisé en utilisant la méthode `dispatchEvent` de l'objet document. Nous aurons donc ce message dans la console : « *Message reçu : Bonjour tout le monde !* ».

Attention **Utiliser les événements depuis le HTML**

```
<button onclick=“maFonction()“>cliquez ici</button>
```

L'utilisation de l'attribut « *onclick* » (ou autre) dans les balises HTML est considérée comme une technique obsolète en JavaScript. Cela est dû à plusieurs raisons, notamment :

- L'attribut « *onclick* » mélange la logique de présentation (HTML) et la logique de comportement (JavaScript), ce qui peut rendre le code difficile à maintenir et à déboguer.
- L'utilisation de l'attribut « *onclick* » peut conduire à des problèmes de sécurité, comme l'injection de code malveillant.
- L'ajout d'événements directement dans le code HTML limite la flexibilité et la réutilisabilité du code.

Au lieu d'utiliser l'attribut « *onclick* » dans les balises HTML, il est préférable d'utiliser les écouteurs d'événements JavaScript pour gérer les événements. Les écouteurs d'événements permettent une séparation plus claire entre la logique de présentation et la logique de comportement. Cela rend le code plus facile à maintenir, à déboguer et à réutiliser.

E. Exercice : Quiz

[solution n°1 p.17]

Question 1

Tous droits réservés à STUDI - Reproduction interdite

Quel est l'objet utilisé pour créer des événements personnalisés en JavaScript ?

- `Event`
- `CustomEvent`
- `MouseEvent`

Question 2

Comment ajoute-t-on un écouteur d'événement pour un événement personnalisé ?

- `addEventListener('nomEvenement', function(){})`
- `on('nomEvenement', function(){})`
- `attachEvent('nomEvenement', function(){})`

Question 3

Comment déclenche-t-on un événement personnalisé ?

- `dispatch('nomEvenement')`
- `trigger('nomEvenement')`
- `dispatchEvent('nomEvenement')`

Question 4

Peut-on inclure des données avec un événement personnalisé ?

- Non, ce n'est pas possible
- Oui, en utilisant l'objet `CustomEvent`
- Oui, en utilisant l'objet `Event`

Question 5

Quel est l'événement JavaScript qui est déclenché lorsqu'un utilisateur appuie sur une touche du clavier ?

- onscroll
- onkeydown
- onmouseover

III. Le fonctionnement avancé des événements

A. L'objet Event

Définition

L'objet `Event` en JavaScript représente un événement qui se produit dans une page web, tel qu'un clic de souris, une pression de touche, un chargement de page, etc. Cet objet contient des informations sur l'événement, telles que le type d'événement, la cible de l'événement, les coordonnées de la souris, etc. Nous recevons une instance de cet objet lors du déclenchement de l'écouteur d'un événement.

Tous droits réservés à STUDI - Reproduction interdite

Voici quelques propriétés et méthodes utiles de l'objet `Event` :

Propriétés :

- type : le type d'événement (ex.: « *click* », « *load* », « *keydown* », etc.),
- target : l'élément qui a déclenché l'événement,
- currentTarget : l'élément qui gère actuellement l'événement,
- clientX, clientY : les coordonnées de la souris par rapport à la fenêtre,
- pageX, pageY : les coordonnées de la souris par rapport au document,
- key : le code de la touche qui a été pressée (pour les événements de type « *keydown* »).

Méthodes :

- `preventDefault()` : empêche le comportement par défaut de l'événement (ex.: empêche le lien de suivre son URL),
- `stopPropagation()` : empêche la propagation de l'événement aux parents de l'élément cible,
- `stopImmediatePropagation()` : empêche la propagation de l'événement aux parents de l'élément cible et empêche l'exécution d'autres gestionnaires d'événements pour cet événement sur l'élément actuel.

Méthode

Voici un exemple de code JavaScript qui écoute l'événement `keyup` pour détecter quand l'utilisateur a appuyé sur la touche « *Entrée* » :

```
const input = document.getElementById('myInput');

input.addEventListener('keyup', function(event) {
  if (event.key === 'Enter') {
    console.log('La touche "Entrée" a été pressée !');
  }
});
```

On utilise ici la variable `Event`, qui est une instance de la classe `Event`. Depuis cette variable, on peut récupérer la propriété « *key* », soit la touche sur laquelle l'utilisateur a cliqué.

B. La propagation des événements

Propagation d'événements

La propagation d'événements est le mécanisme par lequel un événement se propage à travers l'arborescence DOM, de l'élément qui l'a déclenché jusqu'à la racine du document.

Lorsqu'un événement se produit sur un élément, l'agent utilisateur (le navigateur web) vérifie s'il existe un gestionnaire d'événements pour cet événement sur cet élément (le `addEventListener` qu'on aurait pu créer nous-mêmes). Si c'est le cas, il appelle ce gestionnaire d'événements. Ensuite, l'agent utilisateur remonte l'arborescence DOM pour vérifier si d'autres éléments ont également des gestionnaires d'événements pour cet événement. Si c'est le cas, il appelle également ces gestionnaires d'événements, en commençant par l'élément parent immédiat de l'élément d'origine, puis en remontant jusqu'à la racine du document. On va donc de l'élément le plus bas ciblé par l'événement, jusqu'au plus haut, la racine.

Complément **Capture et propagation**

La gestion des événements se déroule en 2 phases que l'on appelle phase de Capture et phase de bouillonnement. La phase de capture se déroule en premier et consiste à traverser le DOM en partant de sa racine jusqu'à l'élément qui est la source de l'événement. Les gestionnaires d'événements rencontrés durant ce parcours sont appelés.

Tous droits réservés à STUDI - Reproduction interdite

La phase de bouillonnement (bubbling) se déroule ensuite et consiste à traverser le DOM en partant de l'élément qui est la source de l'événement en remontant vers la racine du DOM. Les gestionnaires d'événements rencontrés durant ce parcours sont appelés.

Pour ajouter un écouteur d'événements à la phase de Capture, il faut préciser que celui-ci sera exécuté durant la phase de Capture via le paramètre

```
{ capture : true } :

const button = document.querySelector('button');

button.addEventListener('click', () => {
alert('Bouton cliqué');
}, { capture: true });
```

C. La capture d'événements

Définition

La capture d'événements est un mécanisme de gestion des événements qui permet de gérer un événement à un niveau supérieur dans l'arborescence du Document Object Model (DOM).

En règle générale, lorsque vous cliquez sur un élément dans une page web, l'événement de clic est d'abord géré par l'élément lui-même (par exemple là où vous avez cliqué), puis par ses parents dans l'arborescence DOM, jusqu'à ce que l'événement soit propagé à l'ensemble du document.

Méthode

Voici le code à renseigner dans votre fichier HTML :

```
<div id="parent">
  <button id="enfant">Cliquez moi</button>
</div>
```

Voici le code à renseigner dans votre fichier JavaScript :

```
const parent = document.querySelector('#parent');
const enfant = document.querySelector('#enfant');

parent.addEventListener('click', function() {
  console.log('Parent');
});

enfant.addEventListener('click', function() {
  console.log('Enfant');
});
```

Si nous cliquons sur le bouton, nous cliquons aussi sur le div, car le div contient le bouton. Mais, dans ce cas-là, si on clique sur le bouton, nous aurons en console d'abord « *Enfant* », puis « *Parent* ». Car l'événement géré en premier est celui de l'élément le plus bas dans le DOM (ici le button). Après cela, on remonte dans le DOM.

Méthode

Avec la capture d'événements, vous pouvez gérer l'événement à un niveau supérieur de l'arborescence DOM avant qu'il ne soit propagé à l'élément cible. Cela signifie que vous pouvez intercepter l'événement et le gérer avant qu'il n'atteigne l'élément qui a déclenché l'événement. Il suffit de passer « *true* » au troisième argument de la méthode `addEventListener` (capture) pour spécifier que la capture doit être utilisée.

Tous droits réservés à STUDI - Reproduction interdite

```html
<div id="parent">
  <button id="enfant">Cliquez moi</button>
</div>
```

```javascript
const parent = document.querySelector('#parent');
const enfant = document.querySelector('#enfant');

parent.addEventListener('click', function() {
  console.log('Parent');
}, true);

enfant.addEventListener('click', function() {
  console.log('Enfant');
});
```

Dans cet exemple, si vous cliquez sur l'élément enfant, l'événement sera capturé au niveau de l'élément parent avant d'être propagé à l'élément enfant. Vous verrez donc « *Parent* » dans la console avant de voir « *Enfant* ».

En résumé, la capture d'événements permet de gérer les événements à un niveau supérieur de l'arborescence DOM, ce qui vous permet de les intercepter et de les gérer avant qu'ils n'atteignent l'élément cible.

D. Empêcher la propagation d'événements

Méthode

Comme nous venons de le voir, lorsqu'un événement est déclenché sur un élément, il se propage aux éléments parents de l'élément. Cela s'appelle la propagation d'événements. Vous pouvez empêcher la propagation d'un événement en appelant la méthode `stopPropagation()` sur l'objet Event.

```html
<div id="parent">
  <div id="enfant">Cliquez moi</div>
</div>
```

```javascript
const parent = document.querySelector('#parent');
const enfant = document.querySelector('#enfant');

parent.addEventListener('click', function() {
  console.log('Parent');
});

enfant.addEventListener('click', function(event) {
  console.log('Enfant');
  event.stopPropagation();
});
```

Dans ce cas, si vous cliquez sur l'élément enfant, l'événement ne sera pas propagé à l'élément parent. Vous verrez donc seulement « *Enfant* » dans la console.

Si nous enlevons la ligne `'event.stopPropagation();`, alors nous aurons dans la console « *Enfant* », puis « *Parent* ».

Tous droits réservés à STUDI - Reproduction interdite

E. PreventDefault

Exemple

Supposons que vous avez un formulaire d'inscription sur votre site web. Le formulaire comprend des champs pour le nom, l'adresse e-mail et le mot de passe, ainsi qu'un bouton « *S'inscrire* ».

Lorsqu'un utilisateur remplit le formulaire et clique sur le bouton « *S'inscrire* », vous voulez effectuer une validation de formulaire pour vous assurer que les données sont correctes avant de les envoyer au serveur. Pour cela, vous pouvez écouter l'événement de clic du bouton « *S'inscrire* » et empêcher le comportement par défaut du formulaire en utilisant `event.preventDefault()`.

```
const boutonInscription = document.querySelector('#bouton-inscription');
boutonInscription.addEventListener('click', function(event) {
  event.preventDefault();
  // code pour valider les données du formulaire
});
```

Ici, la fonction de gestionnaire d'événements sera appelée chaque fois que l'utilisateur cliquera sur le bouton « *S'inscrire* ». La méthode `event.preventDefault()` empêchera la soumission du formulaire par défaut.

Vous pouvez alors ajouter du code pour valider les données du formulaire. Par exemple, vous pouvez vérifier si l'adresse e-mail est valide ou si le mot de passe est suffisamment sécurisé. Si les données sont valides, vous pouvez les envoyer au serveur *via* une requête AJAX.

Vous pouvez également utiliser des événements pour fournir des commentaires en temps réel à l'utilisateur lorsqu'il remplit le formulaire. Par exemple, vous pouvez ajouter un événement d'entrée pour le champ d'adresse e-mail, qui déclenche une fonction de gestionnaire d'événements pour vérifier si l'adresse e-mail est valide ou non.

```
const champEmail = document.querySelector('#champ-email');
champEmail.addEventListener('input', function() {
  // Vérifier si l'adresse e-mail est valide.
});
```

Dans ce cas, la fonction de gestionnaire d'événements sera appelée chaque fois que l'utilisateur saisira quelque chose dans le champ d'adresse e-mail. Vous pouvez alors ajouter du code pour vérifier si l'adresse e-mail est valide ou non et fournir un feedback en temps réel à l'utilisateur, par exemple en changeant la couleur du champ en fonction de sa validité.

F. Exercice : Quiz

[solution n°2 p.18]

Question 1

Comment fonctionne la capture d'événements en JavaScript ?

- ○ Les événements sont capturés par les éléments parents avant d'être déclenchés sur l'élément cible
- ○ Les événements sont capturés par les éléments enfants avant d'être déclenchés sur l'élément parent
- ○ Les événements sont directement déclenchés sur l'élément cible

Question 2

L'objet `Event` permet de récupérer des informations sur l'événement.

- ○ Vrai
- ○ Faux

Question 3

Tous droits réservés à STUDI - Reproduction interdite

Comment empêcher la propagation d'un événement en JavaScript ?

- En utilisant la méthode `event.preventDefault()`
- En utilisant la méthode `event.stopPropagation()`
- En utilisant la méthode `stopPropagation(event)`

Question 4

Comment capturer un événement en utilisant la phase de capture en JavaScript ?

- En utilisant la méthode `element.addEventListener()` avec le troisième paramètre « *false* »
- En utilisant la méthode `element.attachEvent()`
- En utilisant la méthode `element.addEventListener()` avec le troisième paramètre « *true* »

Question 5

Comment arrêter la propagation d'un événement pendant la phase de capture en JavaScript ?

- En utilisant la méthode `event.stopPropagation()` dans un gestionnaire d'événements capturant
- En utilisant la méthode `event.preventDefault()` dans un gestionnaire d'événements capturant
- En utilisant la méthode `event.stopImmediatePropagation()` dans un gestionnaire d'événements capturant

IV. Essentiel

En résumé, les événements sont des actions qui se produisent dans une application web, comme le clic sur un bouton ou le chargement d'une page. Ils sont gérés à l'aide de gestionnaires d'événements, qui sont des fonctions qui sont appelées lorsqu'un événement se produit, et peuvent être déclenchés de plusieurs manières. L'objet `Event` est utilisé pour gérer nos événements et les variables liées à celui-ci. Nous avons parlé de propagation et de capture d'événement qui correspondent à la manière dont les événements se déplacent dans notre page web.

Pour finir, on peut dire que la gestion des événements est une partie fondamentale de la programmation en JavaScript. Comprendre la capture et la propagation d'événements vous permettra de maîtriser votre développement front, et particulièrement de mieux comprendre les bugs liés aux événements, qui sont récurrents et parfois difficiles à résoudre.

V. Auto-évaluation

A. Exercice

Dans cet exercice, nous allons créer une application de *to-do list* en utilisant la gestion des événements en JavaScript. Notre application doit permettre aux utilisateurs d'ajouter des tâches à la liste, de cocher les tâches terminées et de supprimer les tâches terminées.

Voici le code HTML de base pour notre application :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>To-Do List</title>
    <link rel="stylesheet" href="styles.css"/>
  </head>
  <body>
    <h1>To-Do List</h1>
```


Tous droits réservés à STUDI - Reproduction interdite

```
    <div class="task-form">
        <input type="text" id="new-task" placeholder="Add a new task">
        <button id="add-task">Add Task</button>
    </div>
    <ul id="task-list"></ul>

    <script src="script.js"></script>
  </body>
</html>
```

Voici le JavaScript :

```
const newTaskInput = document.getElementById('new-task');
const addTaskButton = document.getElementById('add-task');
const taskList = document.getElementById('task-list');
```

Et enfin le CSS :

```
/* Style général de la page */
body {
  font-family: Arial, sans-serif;
  margin: auto;
  padding: 0;
  background-color: #f4f4f4;
  max-width: 500px;
}

/* Style de l'en-tête */
header {
  background-color: #333;
  color: #fff;
  padding: 15px;
}

header h1 {
  margin: 0;
  font-size: 36px;
}

/* Style de la liste de tâches */
#task-list {
  list-style-type: none;
  padding: 0;
  margin: 0;
}

#task-list li {
  background-color: #fff;
  padding: 10px;
  border-bottom: 1px solid #ccc;
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin: 10px;
}

#task-list li:last-child {
  border-bottom: none;
}

```

Tous droits réservés à STUDI - Reproduction interdite

```css
#task-list li .task-title {
  flex-grow: 1;
}

#task-list li.completed {
  background-color: #e17777;
}

/* Style du formulaire */
.task-form {
  margin-top: 20px;
  display: flex;
  align-items: center;
}

.task-form input[type="text"] {
  flex-grow: 1;
  padding: 10px;
  font-size: 18px;
  border-radius: 5px;
  border: none;
}

.task-form button {
  background-color: #333;
  color: #fff;
  padding: 10px 20px;
  font-size: 18px;
  border-radius: 5px;
  border: none;
  cursor: pointer;
  margin-left: 10px;
}
```

Question 1

[solution n°3 p.19]

Ajoutez le code pour gérer la création d'une tâche. Au clic sur le bouton « add task », il faut créer une tâche en HTML, et l'ajouter au div qui a pour id « *task-list* ».

Question 2

[solution n°4 p.19]

Ajoutez le code pour ajouter la classe « *completed* » à une tâche lorsqu'on la coche.

B. Test

Exercice 1 : Quiz

[solution n°5 p.19]

Question 1

Quelle méthode permet d'ajouter un écouteur d'événement à un élément HTML ?

- `addEventListener()`
- `createElement()`
- `setAttribute()`

Question 2

Tous droits réservés à STUDI - Reproduction interdite

Comment peut-on empêcher la propagation d'un événement dans l'arbre DOM ?

- `event.preventDefault()`
- `event.stopPropagation()`
- `event.stopImmediatePropagation()`

Question 3

Comment peut-on créer un événement personnalisé en JavaScript ?

- En créant une instance de la classe `CustomEvent`
- `event.dispatch()`
- `event.trigger()`

Question 4

Quelle méthode peut-on utiliser pour retirer un écouteur d'événement d'un élément HTML ?

- `deleteElement()`
- `clearAttribute()`
- `removeEventListener()`

Question 5

Lorsqu'on crée un événement en JavaScript, il est immédiatement déclenché.

- Vrai
- Faux

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. 6 Solution n°1

Question 1

Quel est l'objet utilisé pour créer des événements personnalisés en JavaScript ?

- ○ `Event`
- ◉ `CustomEvent`
- ○ `MouseEvent`

La réponse `CustomEvent` est correcte, car c'est l'objet utilisé pour créer des événements personnalisés en JavaScript. L'objet `CustomEvent` est un sous-type de l'objet `Event`, qui permet de définir des propriétés supplémentaires pour l'événement personnalisé.

Question 2

Comment ajoute-t-on un écouteur d'événement pour un événement personnalisé ?

- ◉ `addEventListener('nomEvenement', function(){})`
- ○ `on('nomEvenement', function(){})`
- ○ `attachEvent('nomEvenement', function(){})`

La réponse `addEventListener('nomEvenement', function(){})` est correcte, car c'est la méthode utilisée pour ajouter un écouteur d'événement pour un événement personnalisé. La méthode `on()` n'est pas utilisée en JavaScript, et `attachEvent()` est obsolète.

Question 3

Comment déclenche-t-on un événement personnalisé ?

- ○ `dispatch('nomEvenement')`
- ○ `trigger('nomEvenement')`
- ◉ `dispatchEvent('nomEvenement')`

`dispatchEvent` est la méthode utilisée pour déclencher un événement personnalisé. Les méthodes `dispatch()` et `trigger()` n'existent pas en JavaScript.

Question 4

Peut-on inclure des données avec un événement personnalisé ?

- ○ Non, ce n'est pas possible
- ◉ Oui, en utilisant l'objet `CustomEvent`
- ○ Oui, en utilisant l'objet `Event`

`CustomEvent` est l'objet qui permet d'inclure des données avec un événement personnalisé. L'objet `Event` ne permet pas d'inclure des données supplémentaires.

Question 5

Quel est l'événement JavaScript qui est déclenché lorsqu'un utilisateur appuie sur une touche du clavier ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ onscroll
- ◉ onkeydown
- ○ onmouseover

L'événement « *onkeydown* » en JavaScript est déclenché lorsqu'un utilisateur appuie sur une touche du clavier. Cet événement peut être utilisé pour détecter les touches spécifiques que l'utilisateur a enfoncées et déclencher des actions en conséquence. Par exemple, si vous souhaitez effectuer une action lorsque l'utilisateur appuie sur la touche « *Entrée* », vous pouvez utiliser la méthode `addEventListener()` pour écouter l'événement « *onkeydown* », puis vérifier si la touche enfoncée est la touche « *Entrée* » en utilisant la propriété « *key* » de l'objet événement. En résumé, l'événement « *onkeydown* » est un événement clé pour la détection des touches du clavier en JavaScript.

Exercice p. 11 **Solution n°2**

Question 1

Comment fonctionne la capture d'événements en JavaScript ?

- ◉ Les événements sont capturés par les éléments parents avant d'être déclenchés sur l'élément cible
- ○ Les événements sont capturés par les éléments enfants avant d'être déclenchés sur l'élément parent
- ○ Les événements sont directement déclenchés sur l'élément cible

Lors de la phase de capture, l'événement est d'abord capturé par l'élément parent de l'élément cible, puis par les parents de cet élément. Finalement, l'événement atteint l'élément cible et est traité par les gestionnaires d'événements associés à cet élément.

Question 2

L'objet `Event` permet de récupérer des informations sur l'événement.

- ◉ Vrai
- ○ Faux

Grâce à l'objet `Event`, nous allons pouvoir récupérer des informations sur l'événement, notamment la cible (par exemple sur un clic, là où a cliqué l'utilisateur).

Question 3

Comment empêcher la propagation d'un événement en JavaScript ?

- ○ En utilisant la méthode `event.preventDefault()`
- ◉ En utilisant la méthode `event.stopPropagation()`
- ○ En utilisant la méthode `stopPropagation(event)`

La méthode `event.stopPropagation()` empêche la propagation de l'événement à travers la phase de propagation. Si un gestionnaire d'événements appelle `event.stopPropagation()`, les gestionnaires d'événements associés aux parents de l'élément cible ne seront pas appelés.

Question 4

Comment capturer un événement en utilisant la phase de capture en JavaScript ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ En utilisant la méthode `element.addEventListener()` avec le troisième paramètre « *false* »
- ○ En utilisant la méthode `element.attachEvent()`
- ◉ En utilisant la méthode `element.addEventListener()` avec le troisième paramètre « *true* »

Le troisième paramètre de la méthode `element.addEventListener()` spécifie si l'événement doit être capturé ou non pendant la phase de capture. En passant « *true* » comme troisième paramètre, vous pouvez capturer l'événement pendant la phase de capture.

Question 5

Comment arrêter la propagation d'un événement pendant la phase de capture en JavaScript ?

- ◉ En utilisant la méthode `event.stopPropagation()` dans un gestionnaire d'événements capturant
- ○ En utilisant la méthode `event.preventDefault()` dans un gestionnaire d'événements capturant
- ○ En utilisant la méthode `event.stopImmediatePropagation()` dans un gestionnaire d'événements capturant

Que vous soyez en phase de capture ou non, l'arrêt de la propagation d'un événement fonctionne de la même manière.

p. 14 Solution n°3

```
addTaskButton.addEventListener('click', function() {
  const taskText = newTaskInput.value;
  const taskItem = document.createElement('li');
  const taskCheckbox = document.createElement('input');
  const taskLabel = document.createElement('label');

  taskCheckbox.type = 'checkbox';
  taskLabel.innerText = taskText;

  taskItem.appendChild(taskCheckbox);
  taskItem.appendChild(taskLabel);
  taskList.appendChild(taskItem);

  newTaskInput.value = '';
});
```

p. 14 Solution n°4

```
taskList.addEventListener('change', function(event) {
  if (event.target.type === 'checkbox') {
    const taskItem = event.target.parentElement;
    taskItem.classList.toggle('completed');
  }
});
```

Exercice p. 14 Solution n°5

Tous droits réservés à STUDI - Reproduction interdite

Question 1

Quelle méthode permet d'ajouter un écouteur d'événement à un élément HTML ?

- [x] `addEventListener()`
- [ ] `createElement()`
- [ ] `setAttribute()`

`addEventListener()` permet d'ajouter un écouteur d'événement à un élément HTML.

Question 2

Comment peut-on empêcher la propagation d'un événement dans l'arbre DOM ?

- [ ] `event.preventDefault()`
- [x] `event.stopPropagation()`
- [ ] `event.stopImmediatePropagation()`

`event.stopPropagation()` permet d'empêcher la propagation d'un événement dans l'arbre DOM.

Question 3

Comment peut-on créer un événement personnalisé en JavaScript ?

- [x] En créant une instance de la classe `CustomEvent`
- [ ] `event.dispatch()`
- [ ] `event.trigger()`

La classe CustomEvent permet de créer un événement personnalisé en JavaScript. Vous pourrez ensuite déclencher cet événement, et mettre des écouteurs sur cet événement.

Question 4

Quelle méthode peut-on utiliser pour retirer un écouteur d'événement d'un élément HTML ?

- [ ] `deleteElement()`
- [ ] `clearAttribute()`
- [x] `removeEventListener()`

`removeEventListener()` permet de retirer un écouteur d'événement d'un élément HTML.

Question 5

Lorsqu'on crée un événement en JavaScript, il est immédiatement déclenché.

- [ ] Vrai
- [x] Faux

Faux. Pour créer un événement, on utilise la méthode `createEvent`. Une fois que l'événement est créé, il faut le déclencher avec la méthode `dispatch`.

Tous droits réservés à STUDI - Reproduction interdite